

VC4 – Descenso del Gradiente y Problemas del Trabajo Práctico

03MIAR – Algoritmos de Optimización

Agenda

- Revisión de la Complejidad. Cálculo de operaciones.
- Descenso del Gradiente
- Problemas del Trabajo Práctico

Problema de la mochila y técnicas voraces

4.2. Problema de la mochila

El **problema de la mochila** o KP (*knapsack problem* en inglés) es otro de los problemas clásicos de optimización combinatoria. Se puede enunciar como la forma de encontrar la mejor elección entre una serie de objetos que tienen asociados un valor y un peso cada uno, de tal manera que el valor total de los objetos sea máximo y el peso no supere un cierto límite.

A pesar de la sencillez del enunciado y de comprensión, el problema es **NP-completo** y, por tanto, su resolución es compleja para tamaños grandes. Ha sido objeto de estudio desde mediados del siglo pasado. Existen dos aspectos que lo convierten en un problema especial destacado en la investigación de operaciones. El primero es que, dada su naturaleza, es un problema para el que no es posible disponer de un algoritmo general que pueda abordarlo, por lo que es necesario tratarlo de manera particular. El segundo aspecto que lo hace tan interesante es que existen muchas situaciones en la vida real que se pueden modelizar como un problema de la mochila.

El problema de la mochila puede resolverse de diferentes maneras:

- Modelizado como problema de **programación lineal entera** (PLE), que veremos en la sección siguiente, dado que consiste en maximizar una función cuyas variables están sujetas a restricciones lineales.
- ~~Técnicas voraces, que vimos en el capítulo anterior.~~
- Algoritmos **genéticos** cuando el tamaño del problema es grande.

Existen **variantes** al problema para adaptarlo a situaciones reales. Una de ellas es el **problema de múltiple elección**, en el que los objetos están agrupados y hay que elegir un representante de cada grupo para incluirlo en la mochila.

Video de Búsqueda_en_Amplitud_Ramificación_y_Poda.mp4



Recuperación en forma de grabación de Búsqueda en amplitud y Ramificación y Poda.

Anuncio

Vistas

Estimado alumno,

He realizado y subido al aula el video con la continuación de la VC3 en la que no dio tiempo a exponer las técnicas de búsqueda en amplitud y Ramificación y Poda. El contenido se pospuso para la sesión AG1 pero por error no se grabó en dicha sesión.

Puedes encontrarlo en : Contenido de la asignatura->Materiales del Docente->Otros Materiales->03MIAR_recording_Busqueda_en_Amplitud_Ramificación_y_Poda.mp4

Saludos

VC4 – Complejidad

03MIAR – Algoritmos de Optimización

Agenda

- Revisión de la Complejidad. Cálculo de operaciones.
- Descenso del Gradiente
- Problemas del Trabajo Práctico

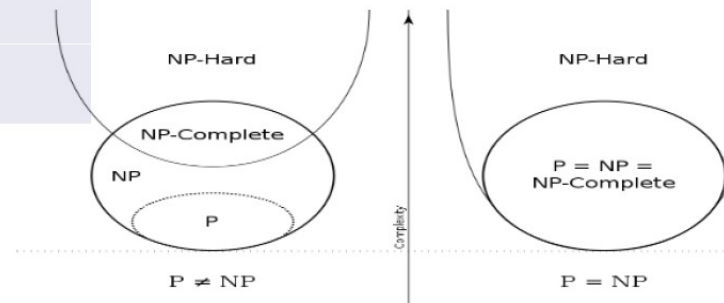
Introducción(Simplificada) a la teoría de la complejidad

- ¿P=NP? O, ¿Qué problemas se pueden resolver fácilmente con computación?

Tipo de problema	Verificable en tiempo P	Solución en tiempo P	Dificultad
P	SI	SI	↓ -
NP	SI	Algunos	
NP-Completos	SI	Desconocido	
NP-Duros	Algunos	Desconocido	+

P= tiempo polinomial

- Problema del millón de dólares

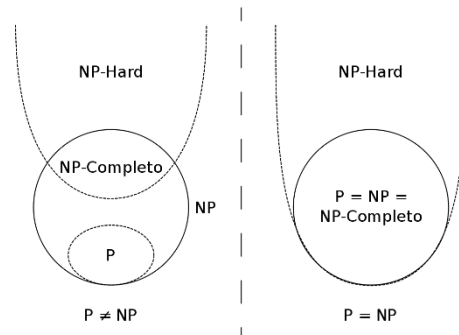


Introducción(Simplificada) a la teoría de la complejidad

• Ejemplos:

- P : Ordenar una lista de números
- NP : Resolver un Sudoku gigante. Es difícil, pero fácil de verificar la solución.
- NP-Completo : Satisfacibilidad booleana(SAT)(1971-Stephen Cook).
 - * Todos los NP-Completo se "*reducen*" a SAT

https://es.wikipedia.org/wiki/Anexo:Problemas_NP-completos
- NP-Duro
 - * Problema del Agente Viajero(TSP)



Introducción(Simplificada) a la teoría de la complejidad

- ¿P=NP?

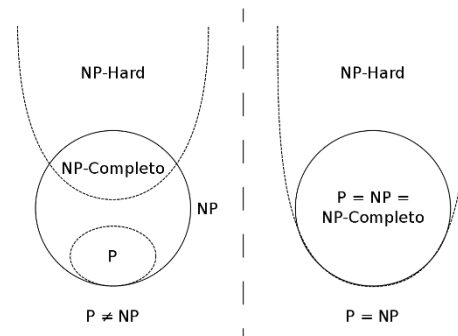
Resolverlo supondría encontrar “atajos” para resolver problemas NP a partir de problemas P (reducción de un problema a otro)

Todo indica que P no es igual a NP por lo que “casi” asumimos que **no podemos** abordar estos problemas computacionalmente(*) con algoritmos exactos!

Es muy probable que solo podamos resolver estos problemas por métodos aproximados o usando heurísticas.



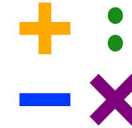
<http://www.claymath.org/millennium-problems/p-vs-np-problem>



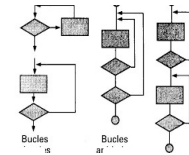
Análisis de Algoritmos

• Contar operaciones elementales

- Suma, resta, multiplicación y división
- Asignaciones
- Condiciones , llamadas a funciones externas y retornos
- Accesos a estructuras de datos
- Ojo con los bucles anidados! 2 bucles $\Rightarrow n^2$

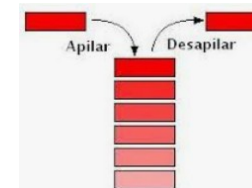


$a+=b$	$a = a + b$
$a-=b$	$a = a - b$
$a*=b$	$a = a * b$
$a/=b$	$a = a / b$
$a\%=b$	$a = a \% b$



• Orden de complejidad

- ✓ Decimos que un algoritmo tiene orden de complejidad $O(f)$ si su tiempo de ejecución en operaciones elementales está acotado, salvo por una constante, por f para todos los tamaños de entrada a partir de un cierto n_0



Análisis de Algoritmos. Sin recursividad

Contar operaciones elementales.

#Fuerza Bruta

```
def distancia_fuerza_bruta(L):
    mejor_distancia = 100000e10
    A,B = (),()
    for i in range(len(L)):
        for j in range(i+1, len(L)):
            D = distancia(L[i],L[j])
            if D < mejor_distancia:
                A,B=L[i],L[j]
                mejor_distancia = D
    return [A,B]
```

distancia_fuerza_bruta(LISTA_2D)

Contar cuantas realiza la función distancia: 2 (en 1-dimensión)

Total Operaciones

Complejidad

$$7 \cdot n \cdot (n-1) \sim n^2$$

$$\text{Total: } 7 \cdot n^2 + 4$$



$$O(n^2)$$

Análisis de Algoritmos. Ecuaciones en recurrencia

El termino n-simo depende de los términos anteriores

```
#Sucesión de Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Calculo del termino n-simo de la sucesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

8

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) - T(n-1) - T(n-2) = 0$$

Ecuación característica:

$$x^2 - x - 1 = 0$$

$$x_1 = \frac{1 + \sqrt{5}}{2} \quad x_2 = \frac{1 - \sqrt{5}}{2}$$

La solución es

$$T(n) = A \cdot \left(\frac{1 + \sqrt{5}}{2} \right)^n + B \cdot \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

A y B dependen de los términos iniciales

Análisis de Algoritmos. Ecuaciones en recurrencia

El termino n-simo depende de una división de los términos anteriores

```
#quick_sort
A = [9187, 244, 4054, 9222, 8373, 4993, 5265, 5470, 4519, 7182, 2035, 3506, 4337, 7580, 2554, 2824, 8357, 4447, 7379]

def quick_sort(A):
    if len(A) == 1:
        return A
    if len(A) == 2:
        return [min(A), max(A)]

    IZQ=[]
    DER=[]

    #pivote = (A[0]+ A[1] + A[2])/3    #Falla en algunos casos, cuando hay valores repetidos y coinciden al principio de alguna lista
    pivote = sum(A)/len(A)           #Buena opción. La mejor opción sería la mediana pero esto pasa por ordenar la lista
                                    # que es precisamente lo que queremos hacer.

    for i in A:
        if i <= pivote :
            IZQ.append(i)
        else:
            DER.append(i)

    return quick_sort(IZQ) + quick_sort(DER)

@calcular_tiempo
def QS(A):
    return quick_sort(A)

print(QS(A))
```

Ecuaciones

$$T_n = \begin{cases} c \cdot n^k, 1 \leq n < b \\ a \cdot T\left(\frac{n}{b}\right) + c \cdot n^k, n \geq b \end{cases}$$

Soluciones

$$T(n) = O(n^k) \text{ si } a < b^k$$

$$T(n) = O(n^k \cdot \log(n)) \text{ si } a = b^k$$

$$T(n) = O(n^{\log_b(a)}) \text{ si } a > b^k$$

$$T(n) = 2 \cdot T(n/2) + n$$

$$a=2$$

$$b=2$$

$$c=1$$

$$k=1$$

En el mejor caso!

Análisis de la complejidad temporal (II)

- Ordenes de complejidad (de menor a mayor)

$O(1)$	Orden constante
$O(\log n)$	Orden logarítmico
$O(n)$	Orden lineal
$O(n \log n)$	Orden cuasi-lineal
$O(n^2)$	Orden cuadrático
$O(n^3)$	Orden cúbico
$O(n^a)$	Orden polinómico
$O(2^n)$	Orden exponencial
$O(n!)$	Orden factorial



Tabla 1

Comparación de los órdenes de complejidad según el tamaño del problema

n	$\log n$	n^2	2^{2n}	$n!$
2	1	4	4	2
8	3	64	256	40.320
32	5	1.024	$4,3 \times 10^9$	$2,6 \times 10^{35}$
100	6	10^4	$1,2 \times 10^{27}$	$9,3 \times 10^{177}$

VC4 – Descenso del Gradiente

03MIAR – Algoritmos de Optimización

Agenda

- Revisión de la Complejidad. Cálculo de operaciones.
- **Descenso del Gradiente**
- Problemas del Trabajo Práctico

Descenso del Gradiente – AGD

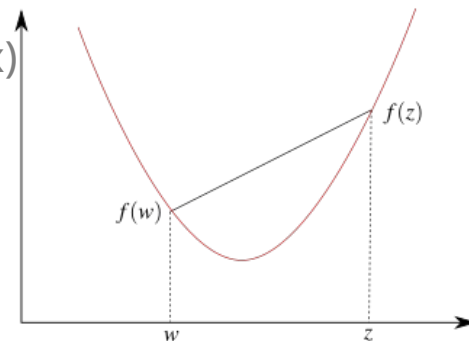
- Es un algoritmo **iterativo** de optimización para encontrar valores mínimos para funciones multi-variables **convexas y diferenciables** (por tanto continuas).

importante

- Convexidad
 - Permitir asegurar que la óptimo que encontremos es global frente a que solo sea local
 - ¿El segmento que une dos puntos está siempre por encima de la función?
 - Si es cóncava y buscamos mínimos, cambiamos a: $-f(x)$



a) Función no convexa



b) Función convexa

Descenso del Gradiente – AGD

- **Diferenciable:** derivable en n variables(dimensiones)
- **Gradiente:** Es un vector(diferente para cada punto = campo vectorial) cuyas coordenadas son las derivadas parciales:

$$\nabla f(\mathbf{r}) = \left(\frac{\partial f(\mathbf{r})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{r})}{\partial x_n} \right)$$

Ejemplo: Dada la función $\mathbf{f(x,y)} = x^2 + 2x + y^2 + y^3 + xy$
el gradiente será:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

$$(2x + y + 2, 2y + 3y^2 + x)$$

En el punto (x,y)

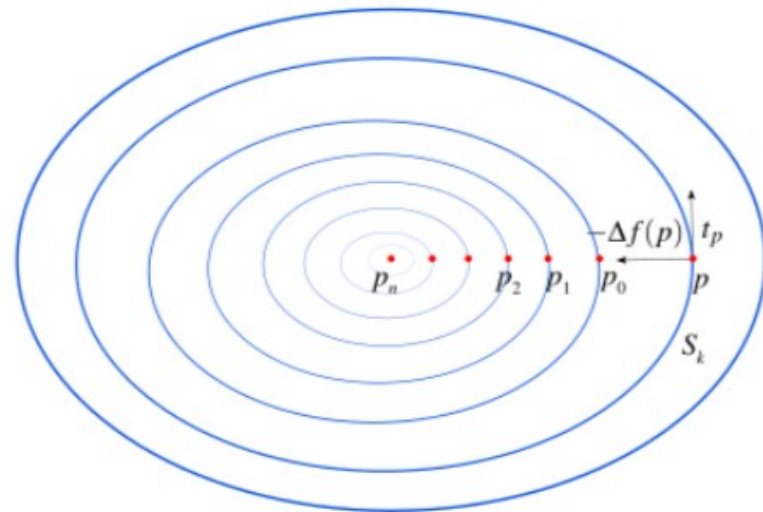
Descenso del Gradiente – AGD

El procedimiento parte de un punto **p** como **solución aproximada(inicial)** y da pasos en el sentido opuesto(si minimizamos) al gradiente de la función en dicho punto.

$$\mathbf{p}_{t+1} = \mathbf{p}_t - \alpha_t \cdot \Delta f(\mathbf{p}_t)$$

La elección de α_t estará condicionada para que :

- $\mathbf{p}_{t+1}$ sea solución factible
- mejore el valor de f respecto a $\mathbf{p}_t$: $f(\mathbf{p}_{t+1}) \leq f(\mathbf{p}_t)$

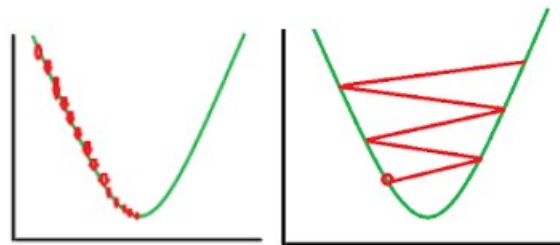


Descenso del Gradiente – AGD

La elección del parámetro α_t , llamado **tasa de aprendizaje (learning rate)**, es importante para hacer efectivo el proceso de acercamiento a la solución óptima.

- Un valor demasiado pequeño puede provocar exceso de iteraciones(Figura 1)
- Un valor demasiado alto puede provocar que el proceso no se acerque lo suficiente(Figura 2)

$$\mathbf{p}_{t+1} = \mathbf{p}_t - \alpha_t \cdot \Delta f(\mathbf{p}_t)$$



learning rate demasiado pequeño
(Figura 1)

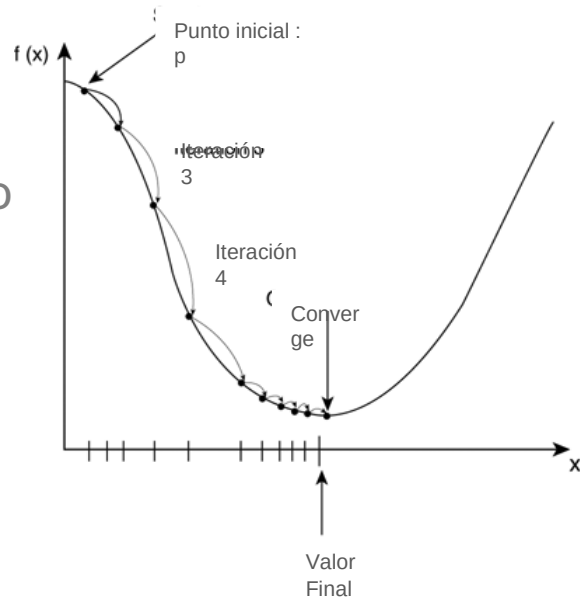
learning rate demasiado grande
(Figura 2)

Descenso del Gradiente – AGD

En general es buena estrategia ir reduciendo el valor de α_t dinámicamente a medida que nos aproximamos a la solución

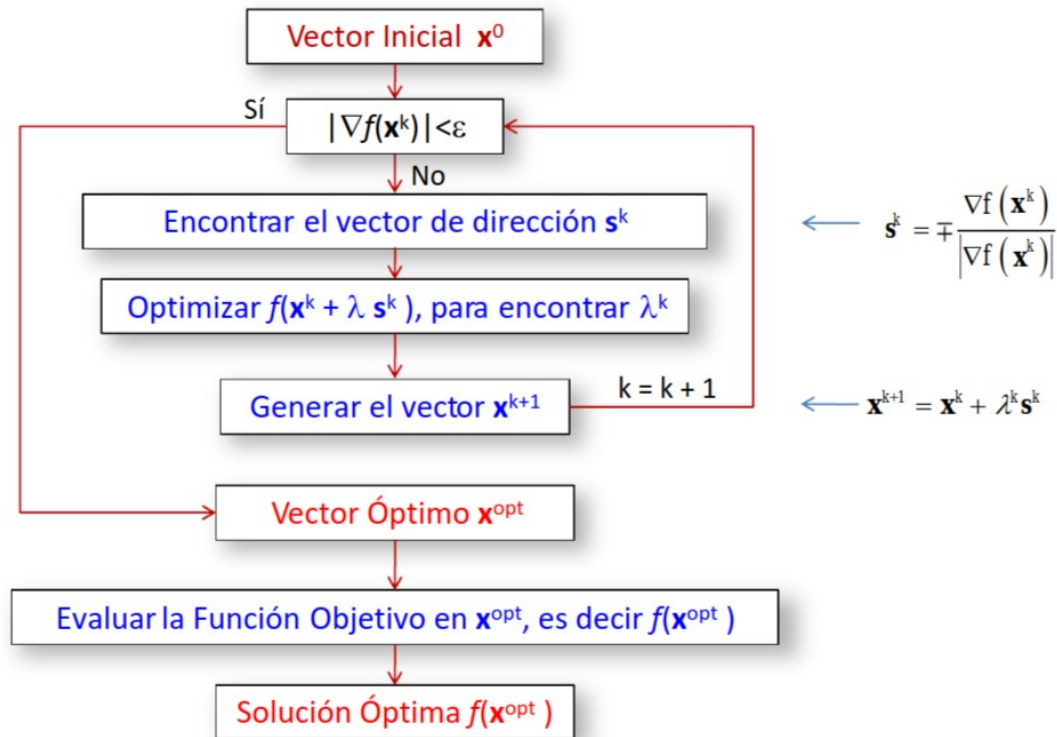
- ¿Como sabemos que nos acercamos?:
 - la magnitud del gradiente
 - cantidad de iteraciones que hemos realizado

$$\mathbf{p}_{t+1} = \mathbf{p}_t - \alpha_t \cdot \Delta f(\mathbf{p}_t)$$



Descenso del Gradiente – AGD

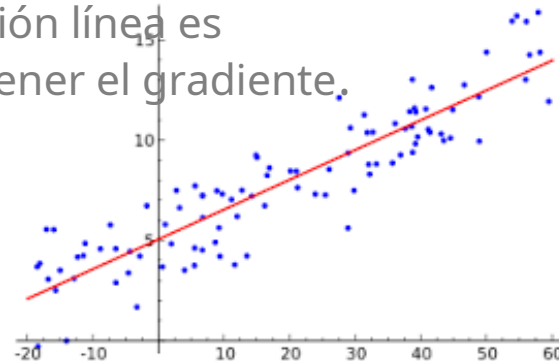
- Diagrama de resolución



Descenso del Gradiente – AGD

- **Aprendizaje supervisado y la regresión lineal**

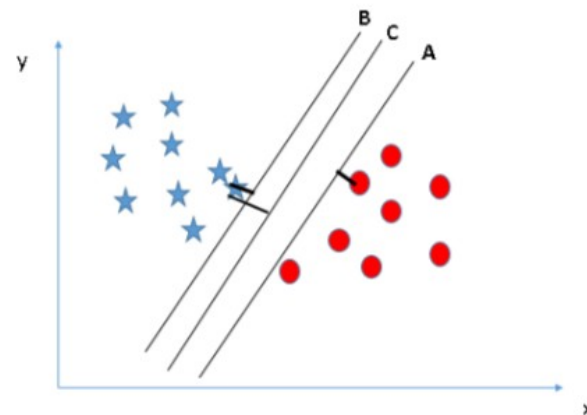
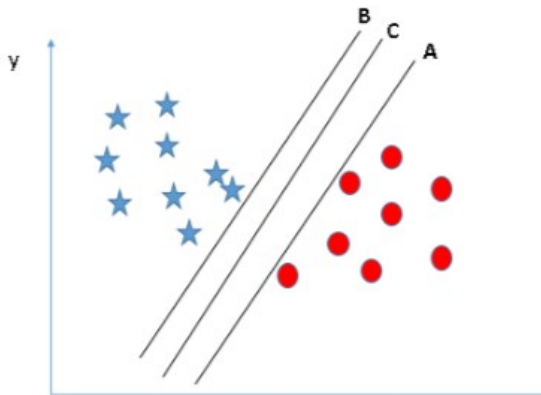
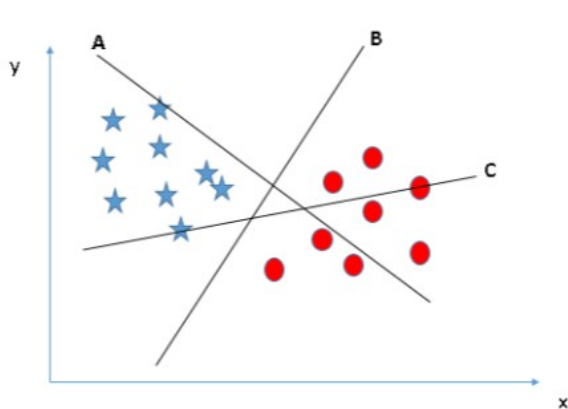
- ✓ En problemas de clasificación tratamos de clasificar los puntos en sus categorías correspondientes de tal manera que se minimice el error.
- ✓ Para la medida de este error solemos utilizar el **error cuadrático medio**(regresión lineal) pero es posible usar otras medidas(por ejemplo: distancia Euclidea, Manhattan, ...)
- ✓ La función que acumula los errores para los valores conocidos la llamamos función de coste. En el caso de la regresión línea es cuadrática, por tanto diferenciable y podemos obtener el gradiente.



Descenso del Gradiente – AGD

- Aprendizaje supervisado y la regresión lineal

¿Cómo identificar la mejor recta (hiperplano en N-dimensión) que separa los conjuntos de datos?

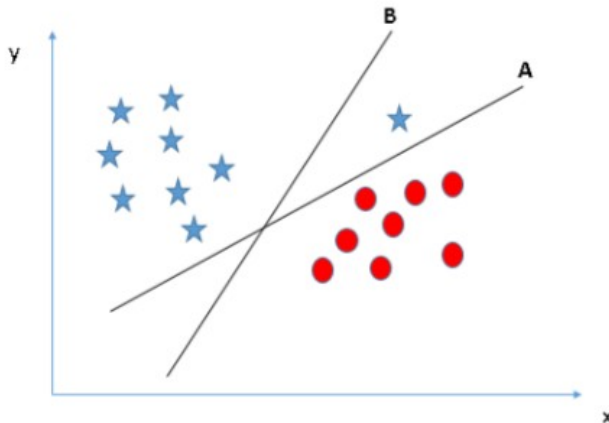


*Seleccionar el que esté
a mayor distancia de ambos conjuntos*

Descenso del Gradiente – AGD

- Aprendizaje supervisado y la regresión lineal

¿Cómo identificar la mejor recta (hiperplano en N-dimensión) que separa los conjuntos de datos?



1º. Seleccionar el que mejor clasifique

2º. Seleccionar el que está a mas distancia

Descenso del Gradiente – AGD

- Aprendizaje supervisado y la regresión lineal

¿Cómo identificar la mejor recta (hiperplano en N-dimensión) que separa los conjuntos de datos?

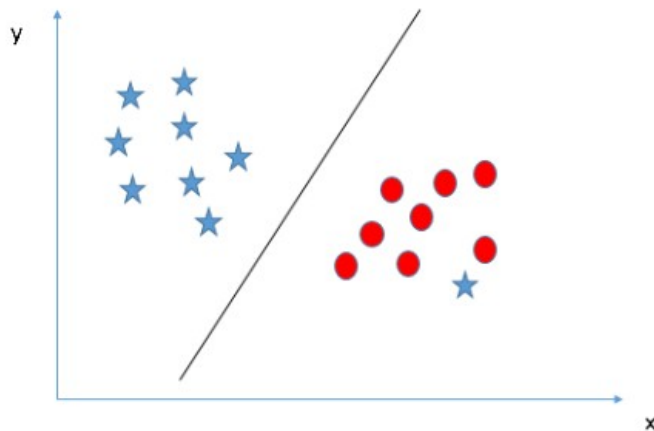


- *No siempre es posible clasificar 100%*
- *¿valores atípicos?*

Descenso del Gradiente – AGD

- Aprendizaje supervisado y la regresión lineal

¿Cómo identificar la mejor recta (hiperplano en N-dimensión) que separa los conjuntos de datos?

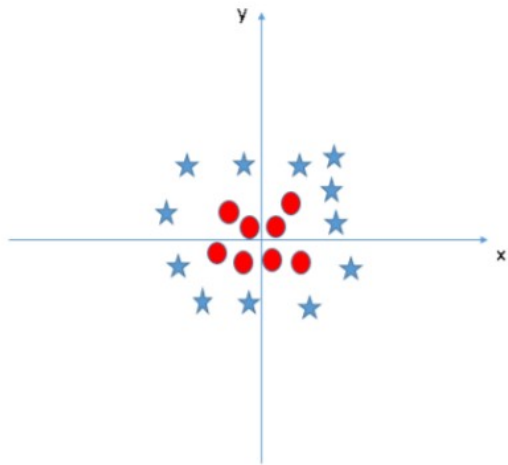


- *Ignorar valores atípicos*

Descenso del Gradiente – AGD

- Aprendizaje supervisado y la regresión lineal

¿Cómo identificar la mejor recta (hiperplano en N-dimensión) que separa los conjuntos de datos?

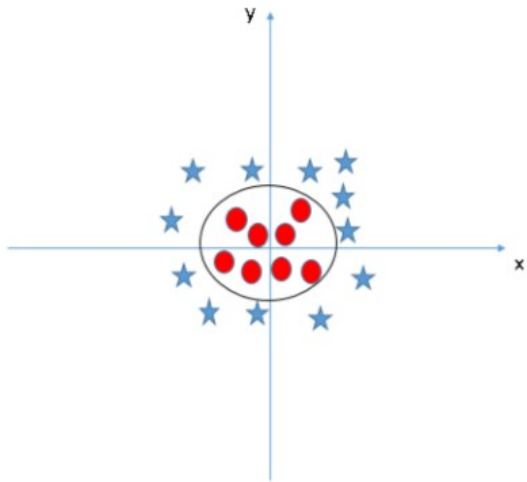


- *No parecen valores atípicos*
- *Parece que si hay una clasificación*
- *Es imposible con hiperplanos lineales*

Descenso del Gradiente – AGD

- Aprendizaje supervisado y la regresión lineal

Podemos extender el procedimiento a funciones no lineales!



Una función como $f(x,y) = x^2 + y^2$ lo resuelve

Descenso del Gradiente – AGD

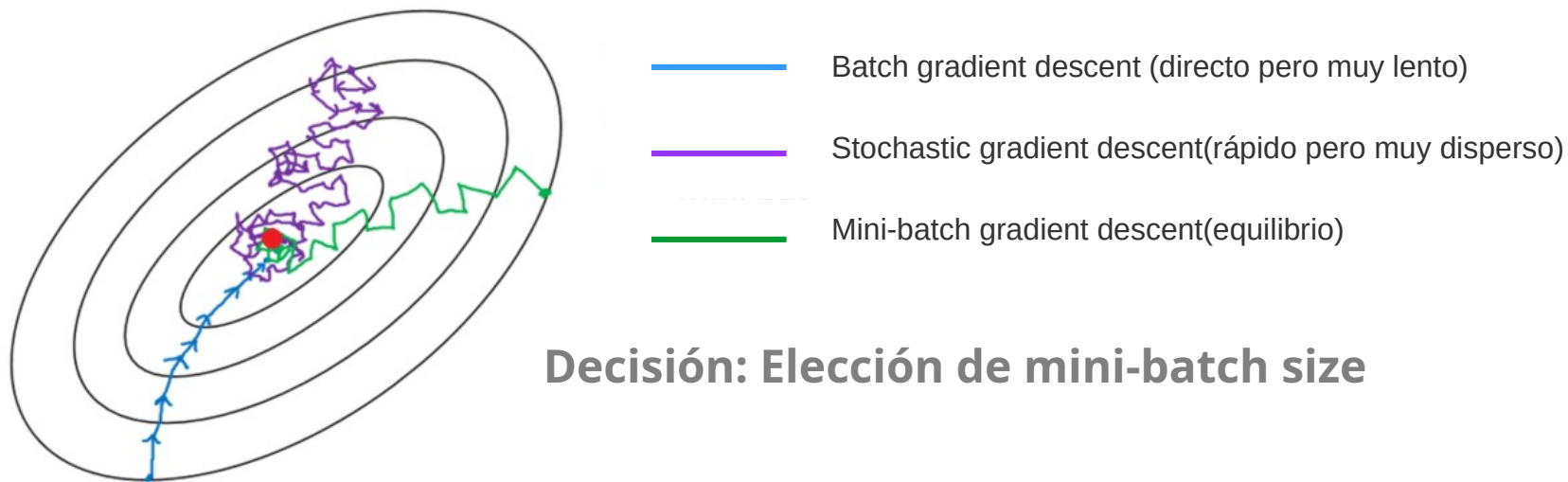
- Otra opción para función de coste(no cuadrática)
- Entropía cruzada(cross-entropy)
 - Para problemas con soluciones binarias
 - **Definición:** Número de bits diferentes. Mide como de diferentes son dos elementos.
 - Usada en algoritmos de clasificación de imágenes

Descenso del Gradiente – AGD

- **Dependiendo del Volumen de Datos**
 - ✓ Descenso del gradiente por lotes (**batch gradient descent**)
 - Calcula la desviación para todos los puntos en cada iteración!!!
 - ✓ Descenso del gradiente estocástico(**stochastic gradient descent**)
 - Calcula la desviación para un punto en cada iteración!!!
 - ✓ Descenso del gradiente por lotes reducido(**mini-batch gradient descent**)
 - Equilibrio entre dos anteriores

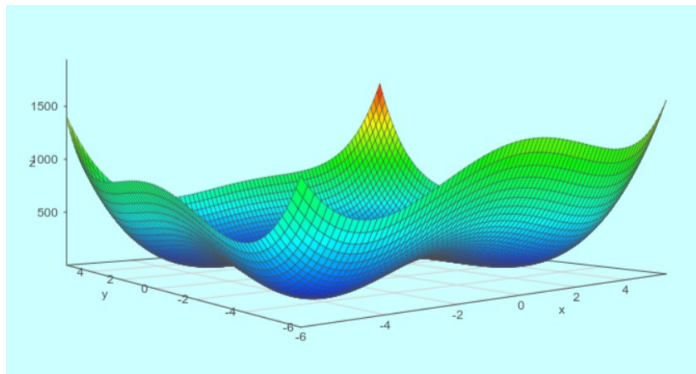
Descenso del Gradiente – AGD

- Dependiendo del Volumen de Datos



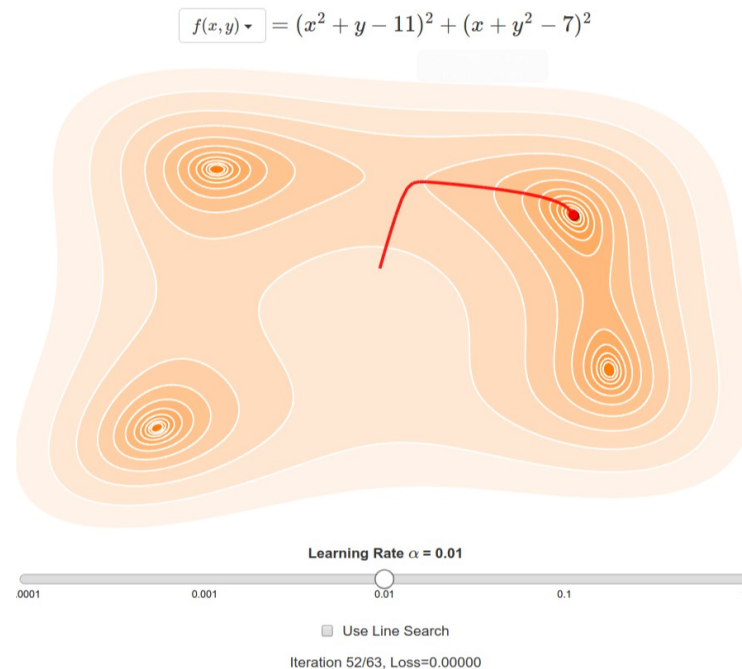
Descenso del Gradiente – AGD

Visualización



$$(x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

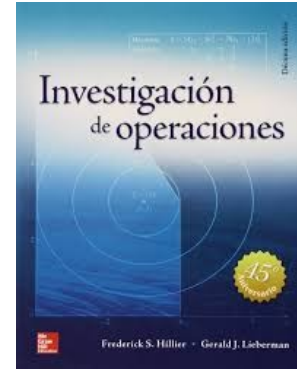
<http://al-roomi.org/3DPlot/index.html>



<https://www.benfrederickson.com/numerical-optimization/>

Ampliación de conocimientos

- ¿Qué es el Descenso del Gradiente? Algoritmo de Inteligencia Artificial | DotCSV
https://www.youtube.com/watch?v=A6FiCDoz8_4
- Hillier, F. S., y Lieberman, G. J. Investigación de Operaciones. Ciudad de México



VC4 – Problemas del Trabajo Práctico

03MIAR – Algoritmos de optimización

Agenda

- Revisión de la Complejidad. Cálculo de operaciones.
- Descenso del Gradiente
- Problemas del Trabajo Práctico

Problema 1. Organizar sesiones de doblaje(I)

- Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible. Los datos son:

Número de actores: 10

Número de tomas : 30

Actores/Tomas : <https://bit.ly/36D8IuK>

- 1 indica que el actor participa en la toma
- 0 en caso contrario

Toma	Actor									
	1	2	3	4	5	6	7	8	9	10
1	1	1	1	1	1	0	0	0	0	0
2	0	0	1	1	1	0	0	0	0	0
3	0	1	0	0	1	0	1	0	0	0
4	1	1	0	0	0	0	1	1	0	0
5	0	1	0	1	0	0	0	1	0	0
6	1	1	0	1	1	0	0	0	0	0
7	1	1	0	1	1	0	0	0	0	0
8	1	1	0	0	0	1	0	0	0	0
9	1	1	0	1	0	0	0	0	0	0
10	1	1	0	0	0	1	0	0	1	0
11	1	1	1	0	1	0	0	1	0	0
12	1	1	1	1	0	1	0	0	0	0
13	1	0	0	1	1	0	0	0	0	0
14	1	0	1	0	0	1	0	0	0	0
15	1	1	0	0	0	0	1	0	0	0
16	0	0	0	1	0	0	0	0	0	1
17	1	0	1	0	0	0	0	0	0	0
18	0	0	1	0	0	1	0	0	0	0
19	1	0	1	0	0	0	0	0	0	0
20	1	0	1	1	1	0	0	0	0	0
21	0	0	0	0	0	1	0	1	0	0
22	1	1	1	1	0	0	0	0	0	0
23	1	0	1	0	0	0	0	0	0	0
24	0	0	1	0	0	1	0	0	0	0
25	1	1	0	1	0	0	0	0	0	1
26	1	0	1	0	1	0	0	0	1	0
27	0	0	0	1	1	0	0	0	0	0
28	1	0	0	1	0	0	0	0	0	0
29	1	0	0	0	1	1	0	0	0	0
30	1	0	0	1	0	0	0	0	0	0



Problema 2. Organizar los horarios de partidos de La Liga(I)

- Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que **maximice la audiencia**.
- Los horarios disponibles se conocen a priori y son los siguientes:

Viernes	20
Sábado	12,16,18,20
Domingo	12,16,18,20
Lunes	20

Problema 2. Organizar los horarios de partidos de La Liga(II)

- En primer lugar se clasifican los equipos en tres categorías según el numero de seguidores(que tiene relación directa con la audiencia). Hay **3 equipos** en la **categoría A**, **11 equipos** de **categoría B** y **6 equipos** de **categoría C**.
- Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos)

	Categoría A	Categoría B	Categoría C
Categoría A	2 Millones	1,3 Millones	1 Millones
Categoría B		0.9 Millones	0.75 Millones

Problema 2. Organizar los horarios de partidos de La Liga(III)

- Si el horario del partido no se realiza a las 20 horas del sábado se sabe que se reduce según los coeficientes de la siguiente tabla
- Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes

	Viernes	Sábado	Domingo	Lunes
12h	-	0.55	0.45	-
16h	-	0.7	0.75	-
18h	-	0.8	0.85	-
20h	0.4	1	1	0.4

Problema 2. Organizar los horarios de partidos de La Liga(IV)

- Es posible la coincidencia de horarios pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

Coincidencias	-%
0	0%
1	25%
2	45%
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

Problema 2. Organizar los horarios de partidos de La Liga(IV)

Los cálculos asociados a una jornada de ejemplo se realizan según se muestra en la siguiente tabla:

Partido	Categorías	Horario	Base(Mill.)	Ponderación	Base*Ponderación	Corrección Coincidencia
Celta - Real Madrid	B-A	V20	1,3	0,4	0,52	0,52
Valencia - R. Sociedad	B-A	S12	1,3	0,55	0,72	0,72
Mallorca - Eibar	C-C	S16	0,47	0,7	0,33	0,33
Athletic - Barcelona	B-A	S18	1,3	0,8	1,04	1,04
Leganés - Osasuna	C-C	S20	0,47	1	0,47	0,47
Villarreal - Granada	B-C	D16	0,75	0,75	0,56	0,42
Alavés - Levante	B-B	D16	0,9	0,75	0,68	0,51
Espanyol - Sevilla	B-B	D18	0,9	0,85	0,77	0,77
Betis - Valladolid	B-C	D20	0,75	1	0,75	0,75
Atlético - Getafe	B-B	L20	0,9	0,4	0,36	0,36

Total: 5,88

$$=0,56 \times 0,75$$

$$=0,68 \times 0,75$$

Problema 3. Generación de Tribunales(I)

- Se precisa configurar tribunales de evaluación para un grupo de 15 alumnos que desean presentar su Trabajo Fin de Máster (TFM).
- Cada tribunal está compuesto por tres profesores, cada uno desempeñando uno de los siguientes roles: Presidente, Secretario o Vocal.
- Los profesores han indicado su disponibilidad horaria para participar en los tribunales de 15h a 21h durante la semana del 15 al 19 de abril:

Número de profesores : 10

Número de tribunales : 15

Disponibilidad/Roles : <https://bit.ly/41QWk8o>

- 1 indica que profesor tiene disponibilidad
- 0 en caso contrario



Problema 3. Generación de Tribunales(II)

Disponibilidad

- 1 indica que profesor tiene disponibilidad
- 0 en caso contrario

#	Dia	15							16							17							18							19						
	Profesor/Hora	15	16	17	18	19	20	21	15	16	17	18	19	20	21	15	16	17	18	19	20	21	15	16	17	18	19	20	21	15	16	17	18	19	20	21
1	RRD	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	1	0	0	1	0	1	0	1	0	1	1	1	1	1	1	1	1	0	0	
2	QYV	1	1	1	1	0	0	0	0	1	1	1	1	0	0	1	0	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	
3	LHL	0	0	1	1	0	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	0	1	0	1	1	0	1		
4	HLC	1	0	1	0	1	1	0	1	0	0	1	1	1	1	0	0	1	1	1	1	1	1	0	1	1	0	1	1	1	1	1	1	0		
5	MSB	1	1	0	1	0	1	1	1	1	1	0	1	1	1	1	0	1	1	0	1	1	0	1	1	1	0	1	1	1	0	1	1	0		
6	PMQ	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	1	1	0	0	1	1	1	1	1	0	1	0		
7	QWF	0	1	1	1	1	1	1	1	1	0	1	1	0	1	0	0	1	1	0	0	1	1	0	0	0	0	1	1	1	1	1	0	1		
8	EBB	1	1	1	1	1	0	0	1	1	0	1	1	1	0	1	1	1	0	0	1	1	0	1	1	1	1	1	1	0	1	1	0	1		
9	IOE	1	0	1	1	0	1	0	0	1	1	1	1	1	1	1	1	0	0	0	1	1	1	1	1	1	1	0	1	0	1	1	1	1		
10	IOA	1	1	0	1	1	0	1	1	0	0	0	0	0	1	1	1	0	0	1	1	1	1	0	0	1	1	1	1	1	1	0	0	0	1	



Problema 3. Generación de Tribunales(III)

Roles

Profesor	ROL
RRD	P,S,V
QYV	P,S,V
LHL	P,V
HLC	S,V
MSB	P,S,V
PMQ	P,S,V
QWF	S,V
EBB	S,V
IOE	P,S,V
IOA	P,S,V



Problema 3. Generación de Tribunales(IV)

- Hay 15 alumnos, por lo que se deben configurar 15 tribunales buscando la configuración más equilibrada posible en cuanto a la cantidad de tribunales asignados a cada profesor, es decir, evitando que un profesor tenga muchos tribunales y otros pocos.
- Obviamente ningún profesor puede asistir a dos tribunales a la misma fecha/hora y no puede ser convocado a un tribunal al que no tiene disponibilidad.

Número de profesores : 10

Número de tribunales : 15

Disponibilidad/Roles : <https://bit.ly/41QWk8o>



Problema del Trabajo Práctico

- Importante!. Además de resolver el problema será necesario responder algunas preguntas relacionadas con:
 - Modelo
 - Análisis
 - Diseño
- Fecha limite de entrega 1ª convocatoria: 6/03/2026
- Fecha limite de entrega 2ª convocatoria: 17/04/2026

Problema del Trabajo Práctico. Rúbrica



- **Modelo**

- ¿Como represento el espacio de soluciones?
- ¿Cual es la función objetivo?
- ¿Como implemento las restricciones?

- **Análisis**

- ¿Que complejidad tiene el problema?. Orden de complejidad y Contabilizar el espacio de soluciones

- **Diseño**

- ¿Que técnica utilizo? ¿Por qué?

Problema del Trabajo Práctico. Entregable



- Generar un Notebook en GitHub (según la plantilla)

Algoritmos de optimización - Trabajo Práctico

Nombre y Apellidos: xxxxxxxx xxxxxxxxxxxx xxxxxxxxxxxx

Url: https://github.com/_/O3MAIR-Algorithmos-de-Optimizaci3n/tree/master/TrabajoPractico

Google Colab: <https://colab.research.google.com/drive/xxxxxxxxxxxxxxxxxxxx>

Problema:

1. Sesiones de doblaje
2. Organizar los horarios de partidos de La Liga
3. Configuraci3n de Tribunales

Descripci3n del problema(copiar enunciado)

....

Modelo

- ¿Como represento el espacio de soluciones?
- ¿Cual es la funci3n objetivo?
- ¿Como implemento las restricciones?

Haz doble clic (o pulsa Intro) para editar

#Respuesta

Análisis

- ¿Que complejidad tiene el problema?. Orden de complejidad y Contabilizar el espacio de soluciones

[1] #Respuesta

Diseño

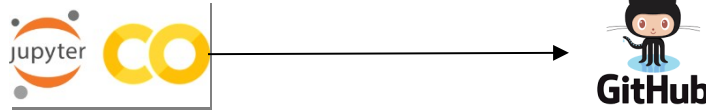
- ¿Que técnica utilizo? ¿Por qué?

#Respuesta

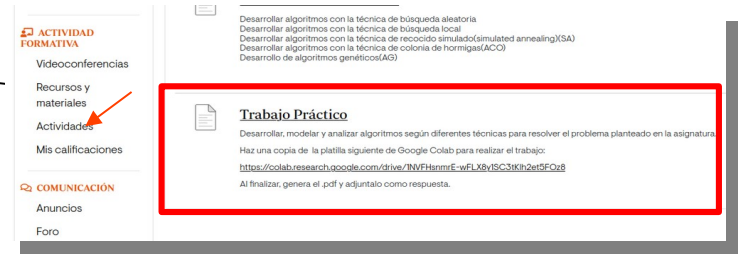
Problema del Trabajo Práctico. Entregable

- Generar un Notebook en GitHub (carpeta TRABAJO_PRACTICO)

30%



- Entrega de documento .pdf con en Notebook (como las A. Guiadas)



Ampliación de conocimientos y habilidades

■ Bibliografía

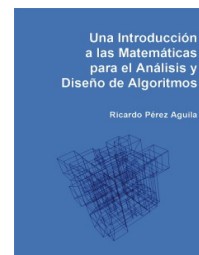
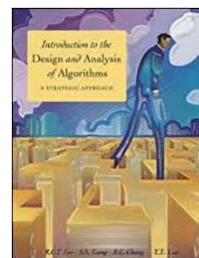
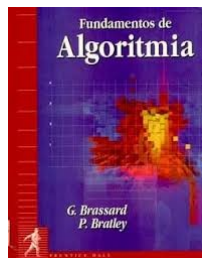
- Brassard, G., y Bratley, P. (1997). Fundamentos de algoritmia. ISBN 13: 9788489660007

- Guerequeta, R., y Vallecillo, A. (2000). Técnicas de diseño de algoritmos.

<http://www.lcc.uma.es/~av/Libro/indice.html>

- Lee, R. C. T., Tseng, S. S., Chang, R. C., y Tsai, Y. T. (2005). Introducción al diseño y análisis de algoritmos. ISBN 13: 9789701061244

- Pérez Aguila, R. (2012). Una introducción a las matemáticas para el análisis y diseño de algoritmos. ISBN 13: 9781413576474. <https://tinyurl.com/yzlt5oed>

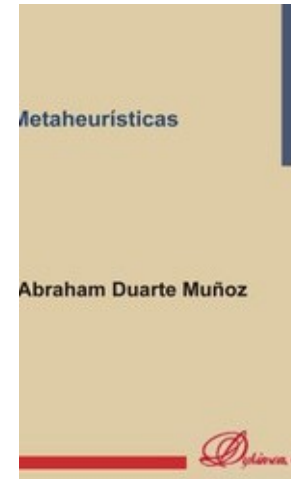
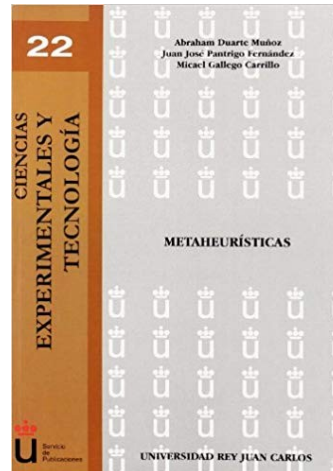


Ampliación de conocimientos y habilidades

■ Bibliografía

- Duarte, A. (2008). Metaheurísticas. Madrid: Dykinson.

<https://elibro-net.universidadviu.idm.oclc.org/es/ereader/universidadviu/35696>



Próximo día, prácticas(AG2) sobre :

- Programación dinámica
- Búsqueda en grafos, ramificación y poda.
- Descenso del gradiente
- Otros(problema de autobuses)

¿Preguntas?



Gracias

raul.reyero@professor.universidadviu.com